

06_ToolAnalysis

July 8, 2026

```
[15]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from consts import *
import numpy as np
from itertools import combinations
from scipy.stats import linregress
from scipy.stats import spearmanr
from scipy.stats import pearsonr
import json
bigfs = 11
smallfs = 9
tinyfs = 7
plt.rcParams.update({"font.size":smallfs})
plt.rcParams.update({"figure.figsize":(16/2.54, 9/2.54)})
sns.set_style("ticks")
sns.set_context("paper")
```

```
[16]: df_p = pd.read_csv("processed_data/
↳2026-07-07_data_processed_participant_withAllIssueWeights.csv")
df_diff = pd.read_csv("processed_data/
↳2026-07-07_data_processed_differences_withAllIssueWeights.csv")
```

1 Reliability

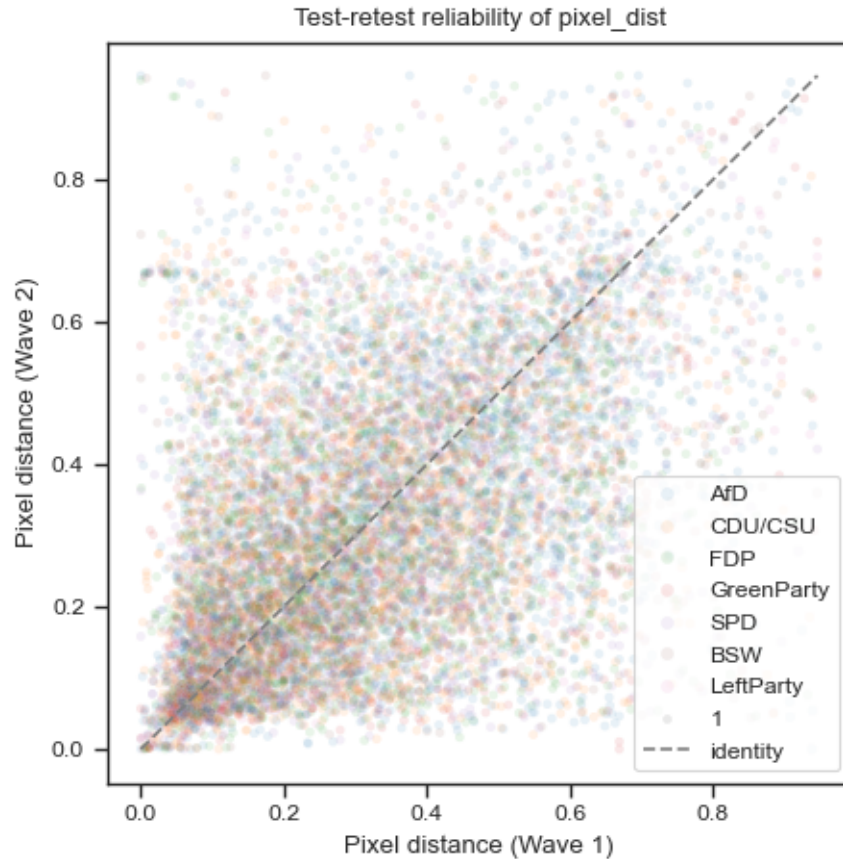
```
[17]: dists_parties = df_diff.loc[((df_diff.dot1=="self") | (df_diff.dot1.
↳isin(partiesVars))) & (df_diff.dot2.isin(partiesVars)) & (df_diff.
↳treatment_wave2==0), ["id", "wave", "dot1", "dot2", "pixel_dist"]].
↳pivot_table(index=["id", "dot1", "dot2"], columns="wave",
↳values="pixel_dist", ).rename(columns={1:"wave 1", 2:"wave 2"}).
↳reset_index().dropna(subset=["wave 1", "wave 2"])
reliability_corrs = dists_parties[["dot1", "dot2", "wave 1", "wave 2"]].
↳groupby(["dot1", "dot2"]).corr().reset_index().query("wave=='wave 2' ").
↳set_index(["dot1", "dot2"])["wave 1"]
reliability_corrs
```

```
[17]: dot1      dot2
      BSW      AfD      0.314768
           CDU/CSU    0.311858
           FDP      0.292822
           GreenParty 0.398607
           SPD      0.436348
      CDU/CSU AfD      0.395701
      FDP      AfD      0.354370
           CDU/CSU    0.261237
      GreenParty AfD      0.486988
           CDU/CSU    0.371325
           FDP      0.382105
           SPD      0.302266
      LeftParty AfD      0.426697
           BSW      0.333035
           CDU/CSU    0.468073
           FDP      0.341766
           GreenParty 0.281329
           SPD      0.327895
      SPD      AfD      0.382650
           CDU/CSU    0.330985
           FDP      0.310142
      self      AfD      0.681680
           BSW      0.546138
           CDU/CSU    0.556335
           FDP      0.472221
           GreenParty 0.691384
           LeftParty 0.623668
           SPD      0.581290
```

Name: wave 1, dtype: float64

```
[18]: fig, ax = plt.subplots(figsize=(5,5))
      sns.scatterplot(data=dists_parties.dropna(subset=["wave 1","wave 2"]),
                      x="wave 1", y="wave 2", alpha=0.1, size=1, ax=ax, hue="dot2")
      lims = [0, dists_parties[["wave 1","wave 2"]].max().max()]
      ax.plot(lims, lims, ls="--", c="gray", label="identity")
      ax.set_xlabel("Pixel distance (Wave 1)")
      ax.set_ylabel("Pixel distance (Wave 2)")
      ax.set_title("Test-retest reliability of pixel_dist")
      ax.legend()
```

```
[18]: <matplotlib.legend.Legend at 0x16af18b00>
```



```
[19]: d_self = dists_parties.loc[dists_parties.dot1=="self"].dropna(subset=["wave_
    ↪1", "wave 2"])
print(f"correlation wave1_mapDistance -- wave2_mapDistance of pairs including_
    ↪self and a typical voter of party A: {pearsonr(d_self["wave 1"],_
    ↪d_self["wave 2"]).statistic:.3f}")
d_all = dists_parties.dropna(subset=["wave 1", "wave 2"])
print(f"correlation wave1_mapDistance -- wave2_mapDistance of pairs including_
    ↪self or a typical voter of party A and a typical voter of party B:_
    ↪{pearsonr(d_all["wave 1"], d_all["wave 2"]).statistic:.3f}")

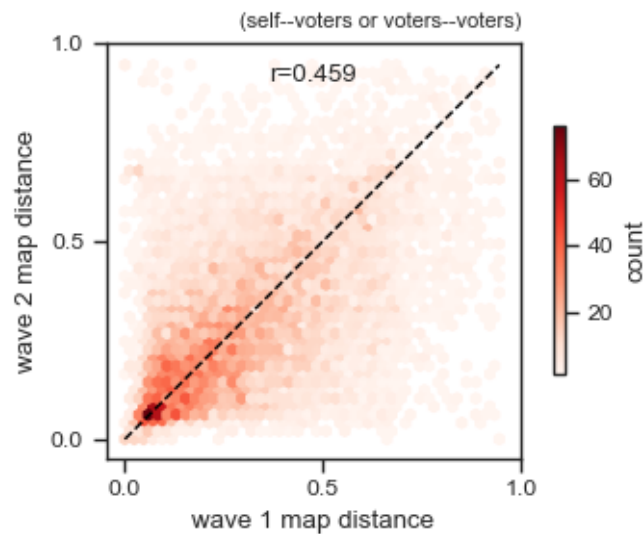
for d in [d_all, d_self]:
    fig, ax = plt.subplots(figsize=(4.,3.))
    hb = ax.hexbin(d["wave 1"], d["wave 2"], gridsize=20 if len(d.dot1.
    ↪unique())==1 and d.dot1.unique()=="self" else 40 ,
        cmap="Reds", mincnt=1)
    lims = [0, dists_parties[["wave 1", "wave 2"]].max().max()]
    ax.plot(lims, lims, ls="--", c="k", lw=1)
    ax.set_xlabel("wave 1 map distance")
    ax.set_ylabel("wave 2 map distance")
```

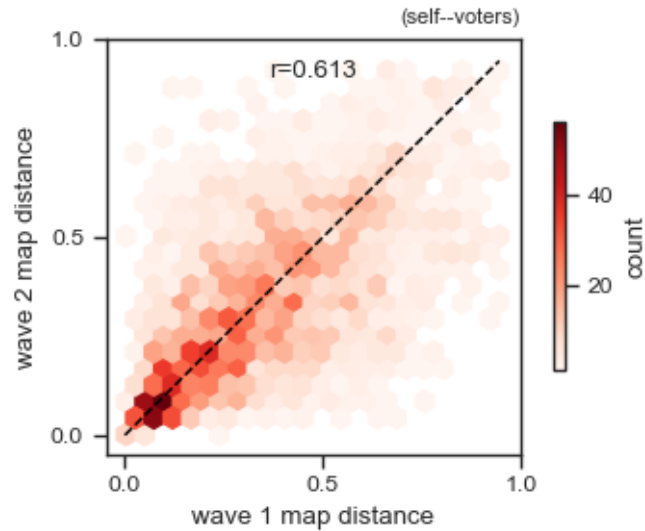
```

ax.set_aspect("equal")
ax.set_xticks([0,0.5,1])
ax.set_yticks([0,0.5,1])
ax.set_title("(self--voters)" if len(d.dot1.unique())==1 and d.dot1.
↪unique()=="self" else "(self--voters or voters--voters)" , x=1,
↪ha="right", fontsize=8)
ax.text(0.5,0.95,f"r={pearsonr(d['wave 1'], d['wave 2']).statistic:.3f}",
↪va="top", ha="center", transform=ax.transAxes)
plt.colorbar(hb, label="count", shrink=0.6)
plt.tight_layout()
plt.savefig(f"figs/reliability_{'self2Voter' if len(d.dot1.unique())==1 and
↪d.dot1.unique()=="self" else '_self2Voter2Voter'}.png", dpi=600)

```

correlation wave1_mapDistance -- wave2_mapDistance of pairs including self and a typical voter of party A: 0.613
correlation wave1_mapDistance -- wave2_mapDistance of pairs including self or a typical voter of party A and a typical voter of party B: 0.459





```
[20]: d_all = dists_parties.dropna(subset=["wave 1", "wave 2"])
d_all["abs_diff"] = (d_all["wave 2"] - d_all["wave 1"]).abs()
mad = d_all["abs_diff"].mean()
scale_range = d_all[["wave 1", "wave 2"]].max().max()
print(f"Mean absolute difference: {mad:.3%} ({mad/scale_range:.2%} of max_
↪ scale)")
```

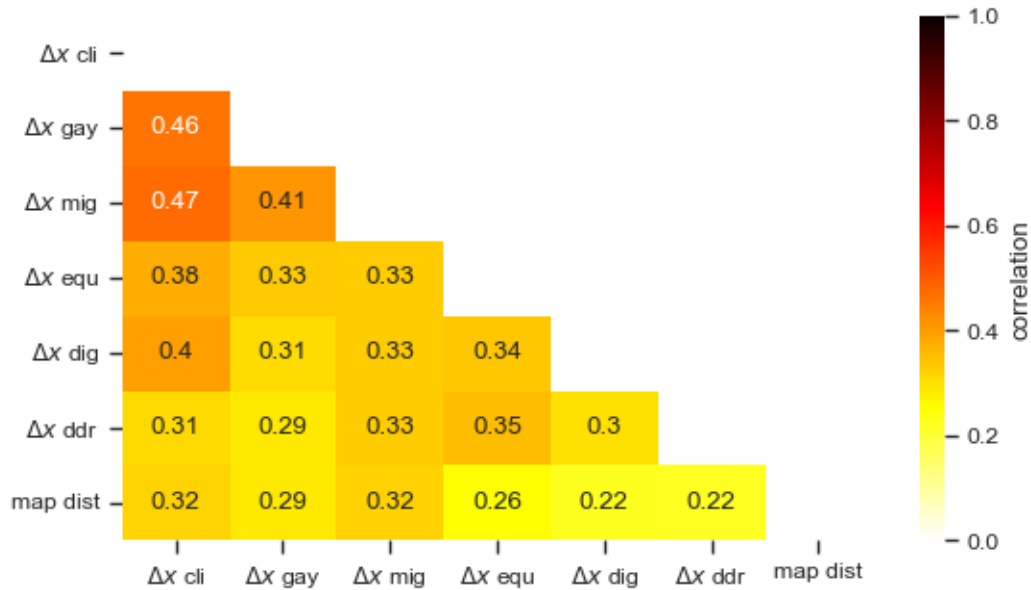
Mean absolute difference: 15.172% (16.05% of max scale)

2 Validitiy

1. Map Distance correlates positively with Opinion Differences

```
[21]: plt.figure()
diff_cols = [f'deltaX_{q}' for q in questions_sc] + ['pixel_dist']
diff_cols_names = [fr'$\Delta x$ {q}' for q in qs] + ['map dist']
mask = np.triu(np.ones_like(df_diff[diff_cols].corr()))
colsNames = dict(zip(diff_cols, diff_cols_names))
sns.heatmap(df_diff[diff_cols].corr().rename(columns=colsNames,
↪ index=colsNames), annot=True, cmap="hot_r", vmax=1, vmin=0, mask=mask,
↪ cbar_kws={'label': "correlation"})
```

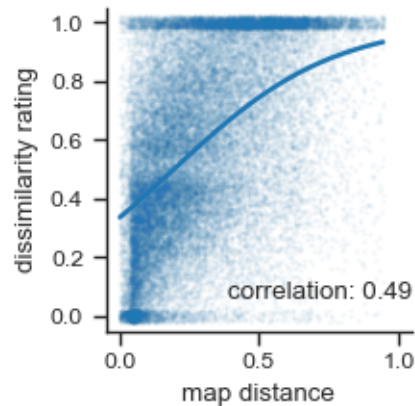
[21]: <Axes: >



2. Map distance correlates positively with dissimilarity

```
[22]: df_diff["pairwise_dissimilarity"] = 1-df_diff["pairwise_similarity"]
sns.lmplot(df_diff, x="pixel_dist", y="pairwise_dissimilarity",
           scatter_kws={"marker": ".", "s": 0.1, "alpha": 0.1 }, y_jitter=0.02, height=6/2.
           ↪54, aspect=1, logistic=True)
plt.ylim(-0.05,1.05)
plt.xlim(-0.05,1.05)
plt.title(f"correlation: {df_diff[['pairwise_dissimilarity']+['pixel_dist']].
           ↪corr().iloc[0,1]:.2f}", y=0.05, x=1, ha="right")
plt.xlabel("map distance")
plt.ylabel("dissimilarity rating")
plt.tight_layout()
plt.savefig("figs/correlation_pairwiseD_map.png", dpi=300)
```

```
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_56007/3689066597.py:1
: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
df_diff["pairwise_dissimilarity"] = 1-df_diff["pairwise_similarity"]
```



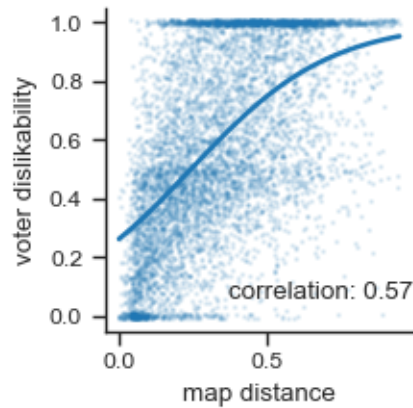
3. Map distance correlates positively with dislike (only voters)

```
[23]: # fig = plt.figure(figsize=(5/2.54,5/2.54))
df_diff["dislikability"] = 1 - df_diff["sympathy"]
print(f"Correlation between dislikability and map distance:␣
↪{df_diff[['dislikability']+['pixel_dist']].corr().iloc[0,1]}")
sns.lmplot(df_diff.loc[df_diff.dot1=="self"], x="pixel_dist",␣
↪y="dislikability", scatter_kws={"marker":".", "s":1, "alpha":0.1 },␣
↪y_jitter=0.01, height=6/2.54, aspect=1, logistic=True)
plt.ylim(-0.05,1.05)
plt.title(f"correlation: {df_diff[['dislikability']+['pixel_dist']].corr().
↪iloc[0,1]:.2f}", y=0.05, x=1, ha="right")
plt.xlabel("map distance")
plt.ylabel("voter dislikability")
plt.tight_layout()
plt.savefig("figs/correlation_sympathyD_map.png", dpi=300)
```

```
/var/folders/07/c88jy3fd1lbwnzsr8qy7w0w0000gp/T/ipykernel_56007/2598799497.py:2
: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
```

```
df_diff["dislikability"] = 1 - df_diff["sympathy"]
```

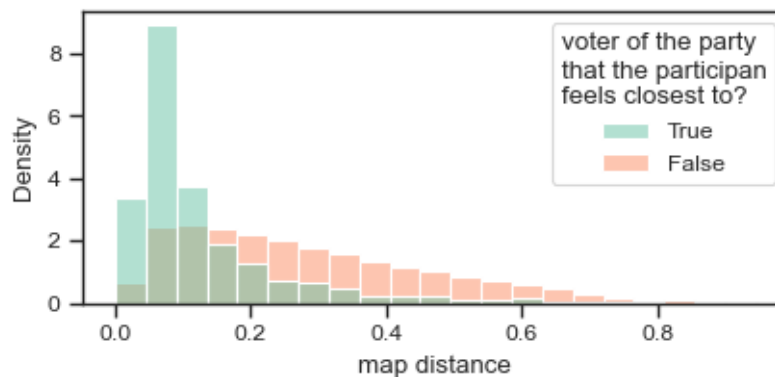
```
Correlation between dislikability and map distance: 0.5677017501802771
```



4. Participants place party they feel closest to close to themselves

```
[24]: plt.figure(figsize=(12/2.54, 5/2.54))
var = "voter of the party\nthat the participan\nfeels closest to?"
sns.histplot(df_diff.rename(columns={"pixel_dist": "map distance",
    ↪ "ingroupdummy": var})
, x="map distance", hue=var, palette="Set2", hue_order=[True, False],
    ↪ stat="density", common_norm=False, bins=21)
```

```
[24]: <Axes: xlabel='map distance', ylabel='Density'>
```



3 Novelty

Different kind of data

```
[25]: wavecondition = "wave in [1]"
fig, axes = plt.subplots(1,2, figsize=(5, 2.), sharey=False, sharex=True)
```

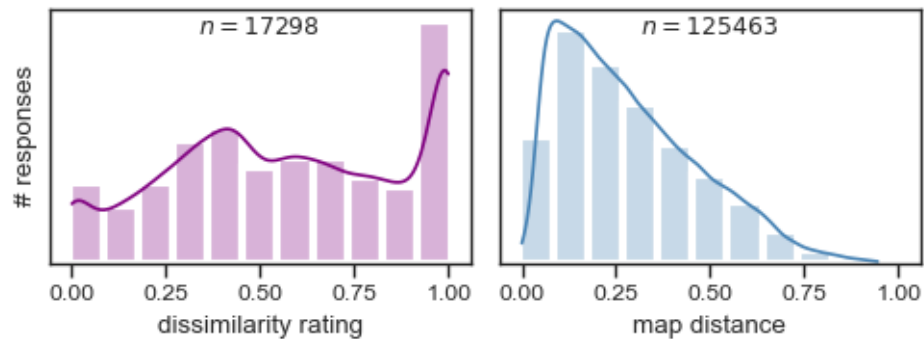


```

df_diff["pairwise_dissimilarity"] = 1-df_diff["pairwise_similarity"]
for ax, col in zip(axes, ["pairwise_dissimilarity", "pixel_dist"]):
    sns.histplot(df_diff.query(wavecondition)[col], color="purple" if "sim" in col
    ↪ else "steelblue", bins=11, binrange=[-0.01,1.01], ax=ax, kde=True,
    ↪ linewidth=3, alpha=0.3)
    ax.set_yticks([])
    ax.set_ylabel(r"# responses" if "sim" in col else "")
    ax.set_xlabel("dissimilarity rating" if "sim" in col else "map distance")
    len_data = df_diff.query(f"wave==2")[col].count()
    ax.text(0.5,0.98, rf"$n={len_data}$", va="top", ha="center", transform=ax.
    ↪ transAxes, fontsize=smallfs)

# df_diff_random = []
# for k, row in df_p.query(wavecondition).iterrows():
#     listpeeps = list(range(row.n_contacts))
#     randpos = np.random.random((2,len(listpeeps)))
#     diffs = [np.linalg.norm(randpos[:,i] - randpos[:,j])/np.sqrt(2) for i, j
    ↪ in combinations(listpeeps, 2)]
#     df_diff_random.extend(diffs)
# ax2 = ax.twinx()
# sns.kdeplot(pd.Series(df_diff_random), ax=ax2, zorder=-1, color="darkgrey",
    ↪ linestyle="--")
# ax2.axis("off")
plt.tight_layout()
plt.savefig("figs/dists_pairwiseDissim_mapDist.png", dpi=600)

```



```

[26]: fig, axes = plt.subplots(1,3, figsize=(7, 2.), sharey=False, sharex=True)
df_diff["dislike"] = 1- df_diff["sympathy"]
for ax, col in zip(axes, ["pixel_dist", "pairwise_dissimilarity", "dislike"]):
    sns.histplot(df_diff.query(f"wave==2 and dot1=='self' and dot2 in
    ↪ {partiesVars}")[col], color="purple" if "sim" in col else ("steelblue" if
    ↪ "pix" in col else "tomato"), bins=11, binrange=[-0.01,1.01], ax=ax,
    ↪ kde=True, linewidth=3, alpha=0.3)

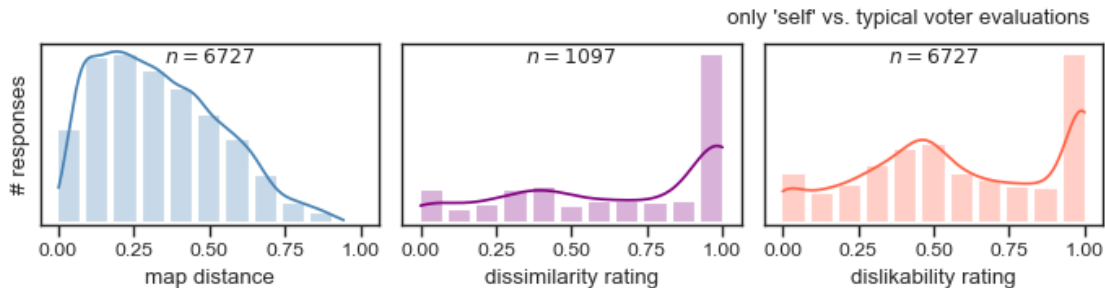
```

```

ax.set_yticks([])
ax.set_ylabel("# responses" if "pixel" in col else "")
ax.set_xlabel("dissimilarity rating" if "sim" in col else ("map distance"
↳if "pix" in col else "dislikability rating"))
len_data = df_diff.query(f"wave==2 and dot1=='self' and dot2 in_
↳{partiesVars}")[col].count()
ax.text(0.5,0.98, rf"$n={len_data}$", va="top", ha="center", transform=ax.
↳transAxes, fontsize=smallfs)
axes[-1].set_title("only 'self' vs. typical voter evaluations", y=1.0, x=0.96,
↳va="bottom", ha="right")
plt.tight_layout()
plt.savefig("figs/dists_pairwiseDissim_mapDist_dislike.png", dpi=600)

```

/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_56007/3931056944.py:2
: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using `pd.concat(axis=1)` instead. To get a
de-fragmented frame, use ``newframe = frame.copy()``
df_diff["dislike"] = 1- df_diff["sympathy"]



```

[27]: fig, axes = plt.subplots(1,4, figsize=(9, 2.), sharey=False, sharex=True)
df_diff["dislike"] = 1- df_diff["sympathy"]
df_diff["socialDistance"] = 1- df_diff["socialCloseness"]
for ax, col in zip(axes, ["pixel_dist", "pairwise_dissimilarity", "dislike",
↳"socialDistance"]):
    sns.histplot(df_diff[col], color="purple" if "sim" in col else ("steelblue"
↳if "pix" in col else ("orange" if "social" in col else "tomato")), bins=11,
↳binrange=[-0.01,1.01], ax=ax, kde=True, linewidth=3, alpha=0.3)
    ax.set_yticks([])
    ax.set_ylabel("# responses" if "pixel" in col else "")
    ax.set_xlabel("dissimilarity rating" if "sim" in col else ("map distance"
↳if "pix" in col else ("dislikability rating [voters]" if "like" in col else
↳"social distance [contacts]")))
    len_data = df_diff[col].count()

```

```

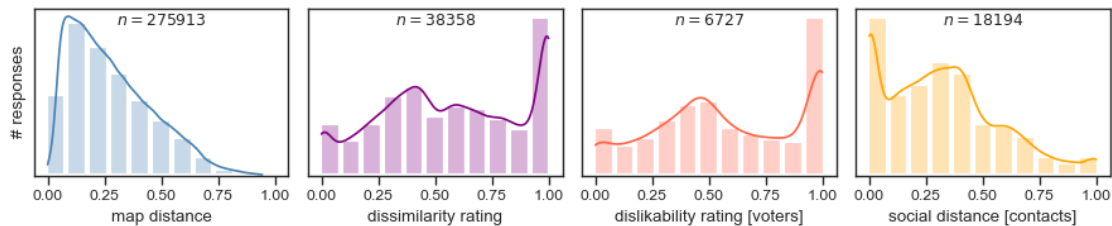
ax.text(0.5,0.98, rf"$n={len_data}$", va="top", ha="center", transform=ax.
↳transAxes, fontsize=smallfs)
plt.tight_layout()
plt.savefig("figs/dists_pairwiseDissim_mapDist_dislike_closeness.png", dpi=600)

```

```

/var/folders/07/c88jy3fd11bdwnzsr8qy7w0w0000gp/T/ipykernel_56007/1123568025.py:3
: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
df_diff["socialDistance"] = 1- df_diff["socialCloseness"]

```



More nuanced dislike/distance patterns

- sympathy question: Participants like their own party and then dislike increases with spectrum distance. There's a left cluster, but otherwise quite strong in-group/out-group story.
- pixel distance: Participants place their own party close, but not necessarily always much closer than others (left cluster).

Diagonal: - AfD seem to dislike AfD the least, but then they place them the furthest away from themselves... Interesting.

Off-diagonal - Lower left triangle: How they see the “more politically right” voters. - Upper right triangle: How they see the “more left” voters - There's an asymmetry!! We dislike and place further away the ones to the left from us!

```

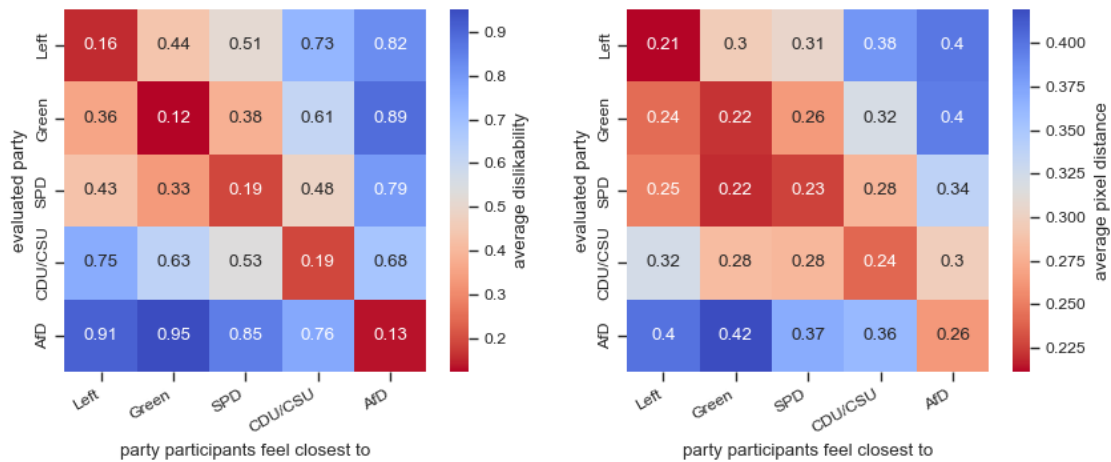
[28]: partiesVarsSelect = [p.replace("Party", "") for p in partiesVars if not
↳p=="FDP" and not p=="BSW"]
fig, (ax1,ax2) = plt.subplots(1,2, sharex=False, sharey=False, figsize=(10,4))
dislike_matrix = 1 - df_diff.loc[df_diff.wave==2].pivot_table(columns="party",
↳index="dot2", values="sympathy", aggfunc="mean")
partyShort = dict(zip(partiesVars, [p.replace("Party", "") for p in
↳partiesVars]))
dislike_matrix = dislike_matrix.rename(columns=partyShort, index=partyShort)
sns.heatmap(dislike_matrix.loc[partiesVarsSelect, partiesVarsSelect],
↳annot=True, cmap="coolwarm_r", cbar_kws={"label":"average dislikability"},
↳ax=ax1)

```

```

dist_matrix = df_diff.loc[df_diff.wave==2].pivot_table(columns="party",
    ↪index="dot2", values="pixel_dist", aggfunc="mean")
dist_matrix = dist_matrix.rename(columns=partyShort, index=partyShort)
sns.heatmap(dist_matrix.loc[partiesVarsSelect, partiesVarsSelect], annot=True,
    ↪cmap="coolwarm_r", cbar_kws={"label":"average pixel distance"}, ax=ax2)
for ax in (ax1, ax2):
    ax.set_aspect("equal")
    ax.set_ylabel("evaluated party")
    ax.set_xlabel("party participants feel closest to")
fig.autofmt_xdate()
plt.savefig("figs/heatmap_dislikeAndDistance_partiesXvoters.png", dpi=600)

```



[]: